2-11-2026

MDRS

Lab2

Victor Manuel Sandoval Maciel
Adolfo Meza Ibarra
Diseño, verificación y validación de sistemas digitales

Content

Entregables. 2
Desarrollo. 3
Diagrama de MDRS_UART. 3
Descripción de módulos. 4
MDRS. 4
UART(General). 6
UART Rx. 7
UART_Tx. 8
General 8
Algoritmos implementados 9
Multiplicación / Square - Booth. 9
División - Restoring. 10
Raíz - Seudocódigo 11
FSM de MDRS_UART. 13
FSM de UART_Rx_Tx. 13
Resultados de Síntesis. 14
Timing con freq máxima 14
Tamaño dathapath 14
Síntesis de datapath y RTL. 15
RTL 15
SignalTab. 15
MDRS Interfaz y Testbench con interfaz ..... **¡Error! Marcador no definido.**
Implementación. 16
Mapeo de pines. 16
Capturas de funcionamiento de MDRS. 16
Multiplicación 16
División 17
Raíz 17
Square 17
Validación 18
Python Script auto-check (Extra 10%). 20
Datapath estructural. 20

Entregables.

Deliverables (at team's github repository):

- A pdf report including:

- Diagrams and explanations of MDRS architecture proposed. Include a functional description of each module. If an FSM is used, include its diagram.
- Simulation screenshots explaining each signal and how MDRS works.
- Synthesis results including Logic utilization and maximum frequency allow for your design.
- Synthesis of the data path where HW limitations are shown in either reports or RTL view.
- FPGA implementation screenshots/log files results.
    - Extra points (10%) Modify python script to automatically check 100% of operations and values.
    - Extra points (20%) Change mdrs_driver_10bits.sv with your own implementation that follows a different strategy.
    - Extra points (20%) Implement the data path in a structural way where units are not larger than:
        - Gates
        - Multiplexors
        - Registers
        - Operations (add/sub/shift/mask)
    - SignalTab screenshot with central register value on it.

- All source files with enough comments.

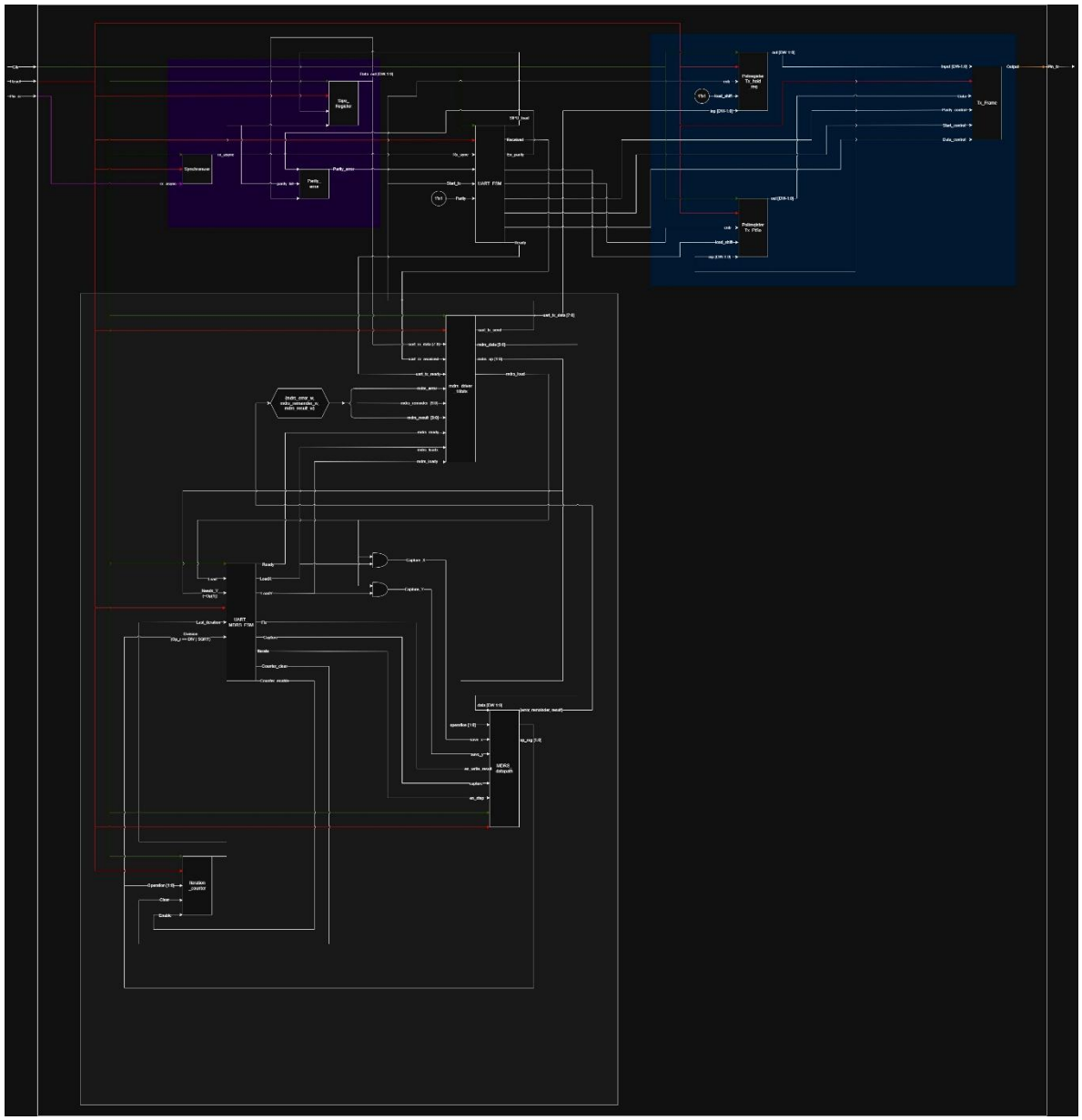
- At least one package must be used for the implementation of the product.

- Testbench file with most important scenarios you considered to test.

- This must use an interface to connect the MDRS block to the testbench.
- It must use at least one class and interacts with MDRS through high-level methods calls in the testbench.
- Extra points (10%) Implement a reference model comparison methodology.
- Extra points (20%) Implement a SVA methodology (verifying all timing relations among load, loadX, loadY, ready and error signals and -1 values in output for error signaling).
- Extra points (30%) Implement a UVM methodology.
- Extra points are considered with the maximum possible value (if UVM + SVA are implemented it takes highest score which is 30%).

Desarrollo.

Diagrama de MDRS_UART.



Descripción de módulos.

MDRS.

- DMRS_Datapath.

Descripción.

Datapath del MDRS para multiplicación, división, raíz y square.
Las cuatro operaciones comparten los registros (X, Y, acumulador, result, remainder, error y operación) y un sumador/restador de 2*DW+1 bits, y sus entradas se eligen con un mux según la operación.
Operación = 0 y 3.
Multiplicación y square: Booth, una iteración por ciclo. X es el multiplicando, Y (o X en square) el multiplicador. El resultado se captura con capture.
Operación = 1.
División restoring se ejecuta sobre valores absolutos. En el ciclo de X se obtiene |X|. Los signos del cociente y del residuo se aplican en el ciclo "en_write_result".
Operación = 2.
La raíz se ejecuta digito por digito, 2 bits de X por iteración (y_reg hace de registro de desplazamiento). En "en_write_result" se corrige el residuo.

Error deja result y remainder en -1 (resultado no representable, división entre 0 o raíz de un negativo).

Parámetros.

DW: Ancho de los datos.

Entradas.

clk:	Reloj del sistema (posedge).
rst:	Reset del sistema (asíncrono y activo en 0).
save_x:	Captura X y la operación (ciclo de LoadX con load).
save_y:	Captura Y (ciclo de LoadY con load).
en_step:	Ejecuta una iteración del algoritmo.
en_write_result:	Escritura final de div y sqrt.
capture:	Captura el resultado de mult y square.
operation [1:0]:	Selecciona la operación (0 = mult, 1 = div, 2 = sqrt, 3 = square).
data [DW-1:0]:	Operando presente.

Salidas.

op_reg [1:0]:	Operación registrada.
result [DW-1:0]:	Resultado de la última operación.
remainder [DW-1:0]:	Residuo de la última operación.
error:	La operación se podrá calcular.

- Sumador_Restador.

Descripción.

Sumador/restador combinacional de DW bits.

Parámetros.

DW: Ancho de los operandos y del resultado.

Entradas.

a [DW-1:0] (signado): Operando A.
b [DW-1:0] (signado): Operando B.

Salidas.

- Result [DW-1:0] (signado): Resultado de la operación.

- Iteration-Counter.

Descripción.

Contador de iteraciones del MDRS. Da la señal de Last_iteration para cuando el conteo llega a la última iteración de la operación en curso.

Entradas.

Clk: Reloj del sistema (actúa de posedge).
Reset: Reset del sistema (activo en 0).
Clear: Limpia el conteo (síncrono, activo en alto, tiene prioridad sobre Enable).
Enable: Incrementa el conteo (activo en alto).
Operation: 0 = mult, 1 = div, 2 = sqrt, 3 = square.

Salidas.

Last_iteration: Ultima iteración de la operación.

- UART_MDRS_FSM.

Descripción.

Máquina de estados de MDRS.

Entradas.

Clk:	Reloj del sistema.
Reset:	Reset activo en 0.
Load:	Señal de carga de operando (activa en alto).
Needs_Y:	La operación necesita segundo operando (para mult o div).
Last_iteration:	El contador llego a la última iteración (activo en alto).
División:	La operación en curso termina con un ciclo de corrección (div o sqrt).

Salidas.

Ready:	El sistema está listo para recibir.
LoadX:	Se espera el operando X.
LoadY:	Se espera el operando Y.
Iterate:	Se tiene una iteración del algoritmo.
Fix:	Escritura final con corrección de signos (para div o el residuo sqrt).
Capture:	Captura de result/remainder/error (para mult y square, al terminar).
Counter_clear:	Limpia el contador de iteraciones.
Counter_enable:	Habilita el contador de iteraciones.

UART(General).

- Uart_fsm.

Descripción.

Maquina de estados Moore de UART (para Rx y Tx). Contiene los contadores de baudios y de bits.
En Rx, el reloj de muestreo se alinea con la mitad del bit:
en “RX_START” se espera la mitad del bit de start, y al cambiar de estado el contador se reinicia, de modo que “baud_tick” cae en la mitad de cada bit
Si en IDLE coinciden Start_tx y el inicio de un frame de Rx, tiene prioridad el Tx para no perder el pulso de envió del driver.

Parámetros.

CLKS_PER_BIT: Ciclos de reloj por bit (clk / baudrate).

Diagrama de estados y transiciones.

Entradas.

Clk:	Reloj del sistema (actúa de posedge).
Reset:	Reset (activo en 0).
Rx_sync:	Línea RX sincronizada.
Parity_error:	Señal de error de paridad del frame recibido.
Start_tx:	Inicio de transmisión (activo en bajo 0).
Parity:	Habilita el bit de paridad en el Tx.

Salidas.

SIPO_load:	Carga el bit muestreado al SIPO (en RX_DATA).
En_parity:	Habilita la comparación de paridad (en RX_PARITY).
Received:	Indica que el frame fue recibido sin error.
load_shift:	Habilita carga paralela (en 1) o desplazamiento (en 0) en PISO.
enb:	Enable de actualización en PISO.
Parity_control:	Control hacia Tx Frame para paridad.
Start_control	Señal de inicio hacia Tx Frame.
Data_control:	Control hacia Tx Frame para datos seriales.
Ready:	Bandera de IDLE (el Tx está disponible).

UART Rx.

- Parity_error.

Descripción.

Calcula la paridad de Data_out según PARITY_TYPE (ODD por default, EVEN opcional) y la compara con el bit de paridad recibido. Si en_parity esta inactivo, parity_error se mantiene en 0.

Parámetros:

PARITY_TYPE:	"ODD" (default) o "EVEN".
DW:	Ancho del bus de datos.

Entradas.

Data_out [DW-1:0]:	Datos recibidos para calcular la paridad.
en_parity:	Habilita la comparación de paridad.
parity_bit:	Bit de paridad recibido.

Salidas.

parity_error: 1 cuando la paridad calculada no coincide con parity_bit.

- SIPO_Register.

Descripción.

Registro SiPo que recibe los datos seriales y los almacena, cuando SIPO_load los datos almacenados salen en paralelo.

Entradas.

clk:	Reloj del sistema (actúa de posedge).
rst:	Reset del sistema.
SIPO_load:	Señal de control que permite el paso de datos al fliflop.
rx_sync:	Entrada serial

Salidas.

Data_out [DW-1:0]:	Bus de salida del módulo (en paralelo).

- Synchronizer.

Descripción.

Sincronizador de 3 flip-flops para la entrada RX.

Entradas.

clk:	Clk del sistema.
rst:	Reset del sistema. (Activo en bajo).
rx_async:	Entrada asíncrona que viene directo del transmisor.

Salidas.

rx_sync:	Salida sincronizada (valor del último flip-flop).

UART_Tx.

- Tx_Frame.

Descripción.

Wrap del frame. Coloca los bits de start, datos y stop según las señales de control de la FSM.

Entradas.

Input:	Bus paralelo de datos [DW-1:0] para cálculo de paridad.
Clk:	Señal de reloj.
Reset:	Reset del sistema (activo en bajo).
Data:	Bit de datos serial proveniente del PISO.
Start_control:	Señal que ordena insertar el start bit (Se activa en 0).
Parity_control:	Habilita el uso de Parity bit

Salidas.

Output: salida del módulo a uart tx.

General.

- MDRS_pkg.

Descripción.

Enumeraciones de operación del MDRS.

- Poli_Register_pkg.

Descripción.

Enumeración de comportamientos para PoliRegister.

- Poli_Register.

Descripción.

Registro secuencial “polimórfico”. Permite por medio de un parámetro, cambiar el ancho de palabra (DW) y su modo de operación.

Entradas.

clk:	Reloj del sistema.
sync_reset:	Reset síncrono, activo en alto.
enb:	Enable de captura del registro.
load_shift:	Selector de modo Load/Shift (1 = Carga paralela, 0 = Desplazamiento).
inp:	Bus de datos de entrada paralelo/serie de tamaño [DW-1:0].

Salidas.

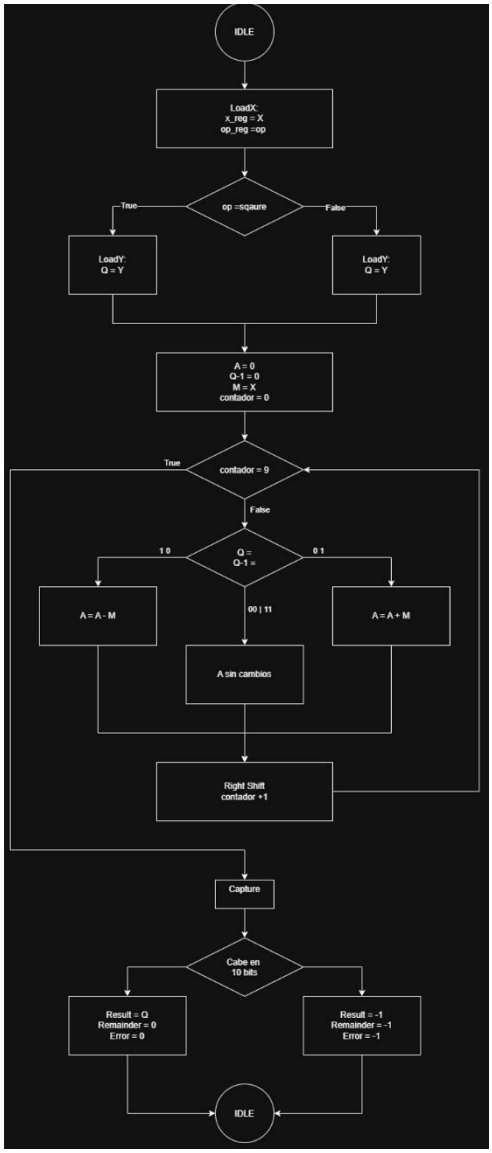
out:	Salida del módulo. Puede sacar el bus paralelo [DW-1:0] o un bit.

Algoritmos implementados

Las implementaciones comparten un acumulador y un módulo sumador/restador

Multiplicación / Square - Booth.

Ambas, multiplicación e Square son secuenciales y utilizan el algoritmo de booth, esta toma DW (Data width o ancho de palabra) ciclos, para este caso son 10.



Variables e registros.

-"x_reg" guarda X y funciona como el multiplicando M.

- Hay un "accumulator" y tiene 21 bits (2·DW+1) y se parte así: [20:11] = A, [10:1] = Q (el multiplicador) y [0] = Q_{-1}. Que es el registro "Product" de Booth.

- "adder_unit", calcula A $\pm$ M. Su entrada se elige por "op_now".

Mult y square caen en el default del mux:

"adder_a" = A con signo extendido.

"adder_b" = M con signo extendido.

"adder_sel" = "Subtract_w".

La decisión de Booth sale de "Product" [1:0], o sea {Q_0, Q_{-1}}:

Q y Q_{-1}	El acumulador…
0 1	Se le suma X
1 0	Se resta X
1 1, 0 0	Queda igual

- "Product_next" = { "acc_next" [10:0], "Product" [10:1] } es el desplazamiento aritmético a la derecha. El bit 10 de la suma de 11 bits queda como signo de A.

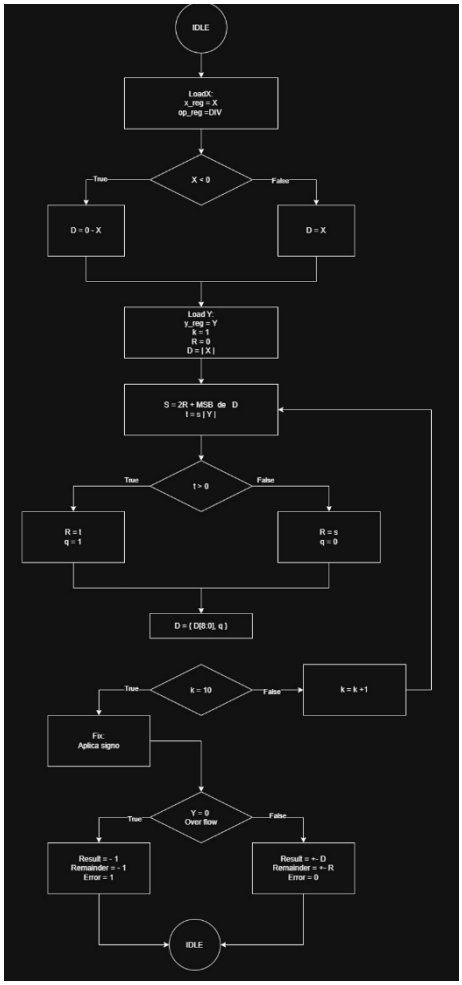
A través de los ciclos podemos describir el cálculo como

Ciclo	Estado	
-	IDLE	Load con X y "save_x"=1. "op_now" toma "operation" porque "op_reg" aún no está escrito. Y "x_reg" = X, "op_reg" = "op" y "accumulator" = {0, X, 0}. Como "Needs_Y" = ~op[1] = 1, la FSM pasa a WAIT_Y.
-	WAIT_Y	Espera Load. Mientras no llegue, no cambia nada. Al llegar Y, "save_y" = 1 hace "Load" = 1 y "accumulator" = {0, Y, 0}. Esto pisa lo que escribió en IDLE y deja X en x_reg como M y Y en Q como multiplicador. La FSM pasa a RUN.

0-8	RUN	Con “en_step” =1, “accumulator” = “Product_d_w” = “Product_next”. Cada ciclo hace una decisión Booth (sumar, restar o nada) y un shift a la derecha. El contador pasa de 0 a 9.
9	RUN	“Iterate” = 0, así que el registro no se actualiza. En este mismo ciclo, de forma combinacional, se hace la iteración 10 y se captura “Product_next” en “result”.
-	IDLE	Ready=1. El driver manda por UART result, remainder y error.

Para los casos en donde la operación no cabe Overflow se calcula sobre “Product_next” [20:10], 11 bits. Si son todos 0 o todos 1, el producto cabe en 10 bits con signo (“Overflow_w” = 0). Con esto se tendría.
result = -1 (todos unos), remainder = -1 y error = 1.
Si no, hay overflow, “result” = “Product_next” [10:1] (el valor de Q), remainder = 0 y error = 0.

División - Restoring.



Registros y sumador en división
“x_reg” guarda X, y también “x_sign”, que se usa en FIX.
“y_reg” guarda Y, el divisor, que se escribe en el ciclo de save_y.
“accumulator” (de 21 bits) se parte: [20:10], en el residuo parcial R (11 bits) y [9:0] es el dividendo restante junto con el cociente.
Cada ciclo se mete un bit del cociente por abajo y sale un bit del dividendo por arriba. El siguiente bit del dividendo siempre es “accumulator”[9].
El sumador calcula “shifted_remainder” – Y en 11 bits, con “shifted_remainder” = {R[9:0], accumulator[9]}.

Si Y > 0, “sel” = ~”y_sign” = 1 y hace shifted − Y.

Si Y < 0, “sel” = 0 y hace shifted + Y, que es lo mismo que restar Y.
“divisor_fits” = ~”adder_sum”[10] indica si la resta no dio negativa.

La actualización es restoring.
Si cabe: “acc” = {sum[10:0], acc[8:0], 1}, o sea el residuo es la resta y el bit del cociente es 1.
Si no cabe: “acc” = {shifted, acc[8:0], 0}, o sea el residuo se restaura y el bit del cociente es 0.

Ciclo	Estado	
-	IDLE	"Load" con X y "save_x" = 1. "op_now" toma "operation", "x_reg" = X y "op_reg" = "DIV". El sumador calcula 0 – X, si X es negativo ("sel" = data[9]), y se guarda "acc" = {0, 0, X}. Como "Needs_Y" = 1, la FSM pasa al estado WAIT_Y.
-	WAIT_Y	Espera Load. Al llegar Y, divisor = data y "y_reg" = Y. En este mismo ciclo se hace la iteración 1: "shifted" = {0, |X|[9]}, se resta |Y| y se actualiza el acumulador según "divisor_fits". La FSM pasa a RUN.
0-8	RUN	Se hace una iteración restoring por ciclo, con divisor = "y_reg". El contador cuenta de 0 a Last_div = DW−2 = 8. En R8 la FSM ve Last_iteration y Division=1, así que sigue a FIX.
9	FIX	Se aplican signos y se escribe result, remainder y error.
-	IDLE	Ready=1. Los resultados ya están estables en los registros.

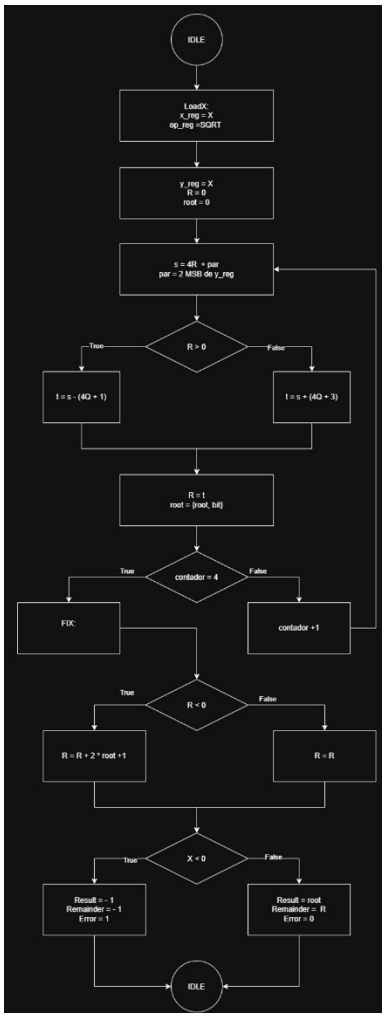
Raíz - Seudocódigo

x_reg guarda X, que solo se usa para su signo en FIX.
y_reg guarda una copia de X y funciona como registro de desplazamiento. Cada iteración usa sus 2 bits más altos y luego se recorre 2 posiciones a la izquierda.
accumulator (21 bits) se parte así:
[20:10] es el residuo R con signo y [9:0] es la raíz Q construida hasta ahora. El bit 20 es el signo de R.
Cada iteración usa el sumador único:

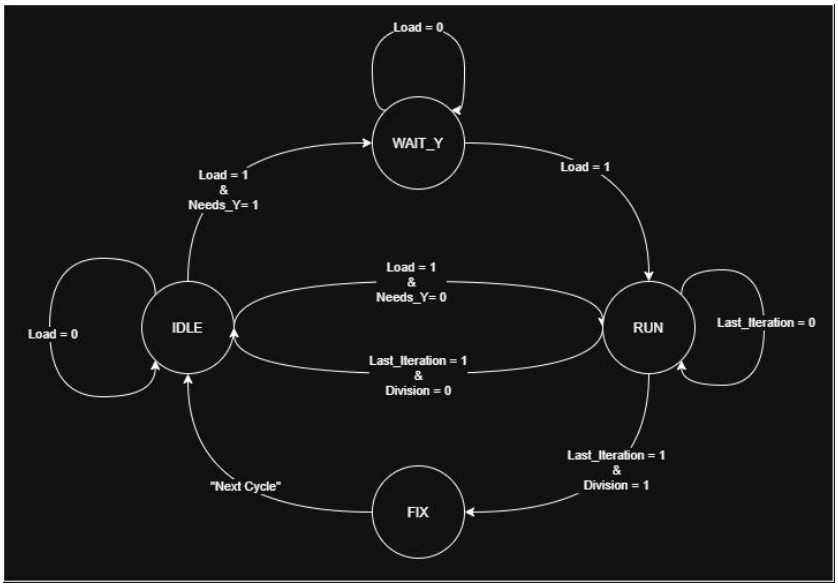
a = 4R + par: {acc[18:10], par}.
b = 4Q + 1 si R ≥ 0, o 4 Q + 3 si R < 0: {root[8:0], signo_R, 1}.
sel = ~signo_R: si R ≥ 0 resta, si R < 0 suma.

El bit nuevo de la raíz es root_bit = ~adder_sum[10]
acc = { adder_sum[10:0], root[8:0], root_bit }.
R pasa a ser la suma o resta, y la raíz se recorre 1 bit metiendo el bit nuevo por abajo.

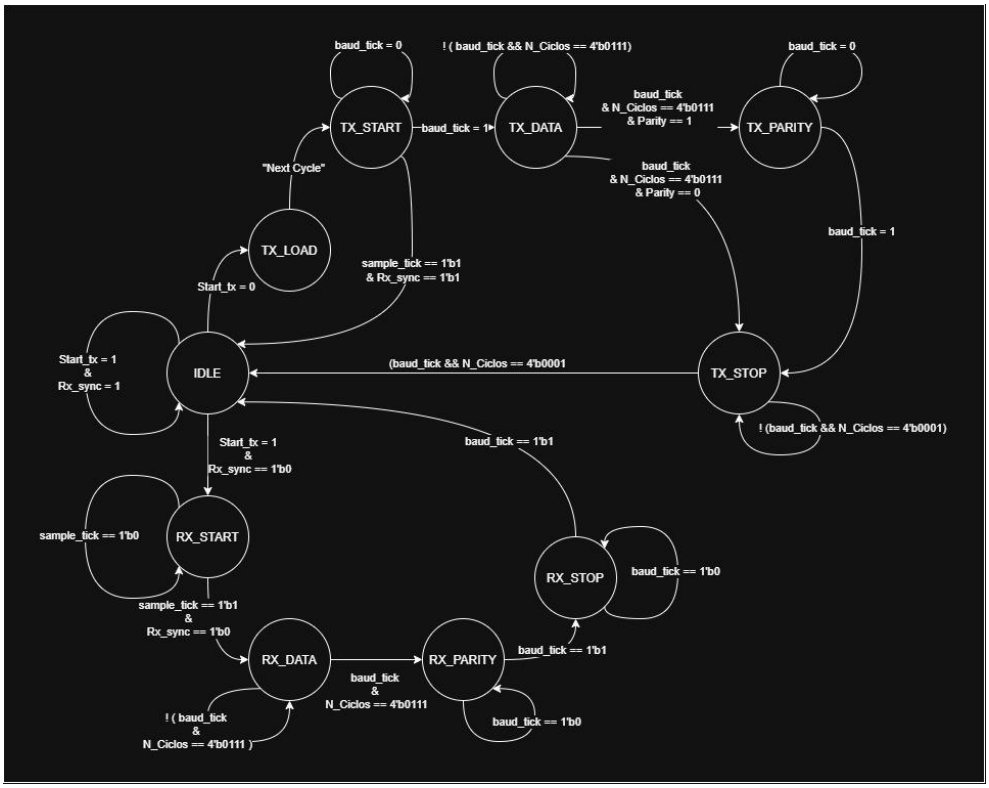
Ciclo	Estado	
-	IDLE	Load con X y save_x=1. Se hace x_reg = X, op_reg = SQRT, y_reg = X y acc = 0. Como Needs_Y = ~op[1] = 0, la FSM salta WAIT_Y y va directo a RUN.
0-4	RUN _Y	Una iteración por ciclo, 2 bits de X cada una. El contador va de 0 a Last_sqrt = DW/2 − 1 = 4. En el ciclo 4 la FSM ve Last_iteration y Division=1, así que sigue a FIX.
5	FIX	Si R quedó negativo, el sumador hace R + 2·root + 1. Si X era negativo, escribe error.
-	IDLE	Ready=1. result, remainder y error listos para otra operación.


IDLE
LoadX:
x_reg = X
op_reg =SQRT
y_reg = X
R = 0
root = 0
s = 4R + par
par = 2 MSB de y_reg
R > 0
True
False
t = s - (4Q + 1)
t = s + (4Q + 3)
R = t
root = {root, bit}
contador = 4
True
False
FIX:
contador +1
R < 0
True
False
R = R + 2 * root +1
R = R
X < 0
True
False
Result = - 1
Remainder = - 1
Error = 1
Result = root
Remainder = R
Error = 0
IDLE

FSM de MDRS_UART.

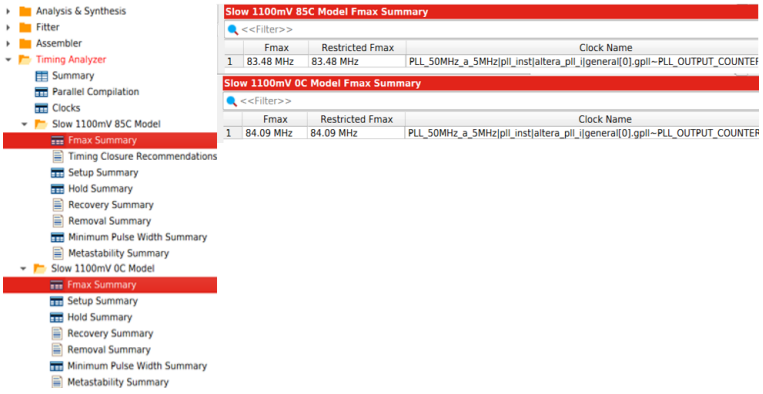


FSM de UART_Rx_Tx.



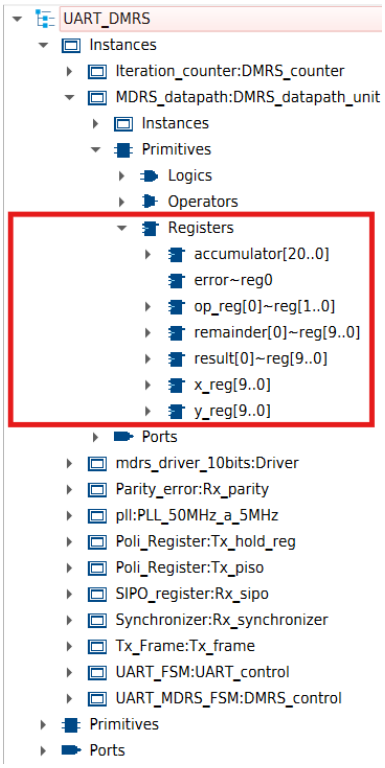
Resultados de Síntesis.

Timing con freq máxima



Según el reporte de Timing, la frecuencia máxima que deberíamos usar es 83 MHz, por lo que usando 5 MHz estamos bastante bien.

Tamaño dathapath

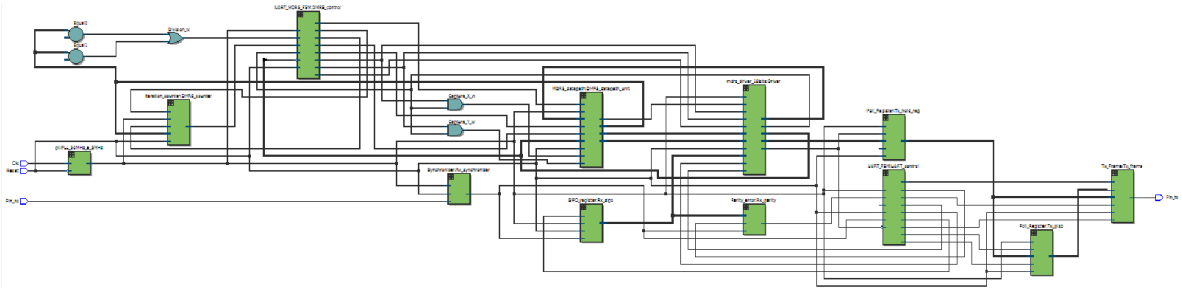


Nombre del registro/s	N. Registros(FlipFlops)
accumulator	21
error	1
op_reg	2
remainder	10
result	10
x_reg	10
y_reg	10
Total	64

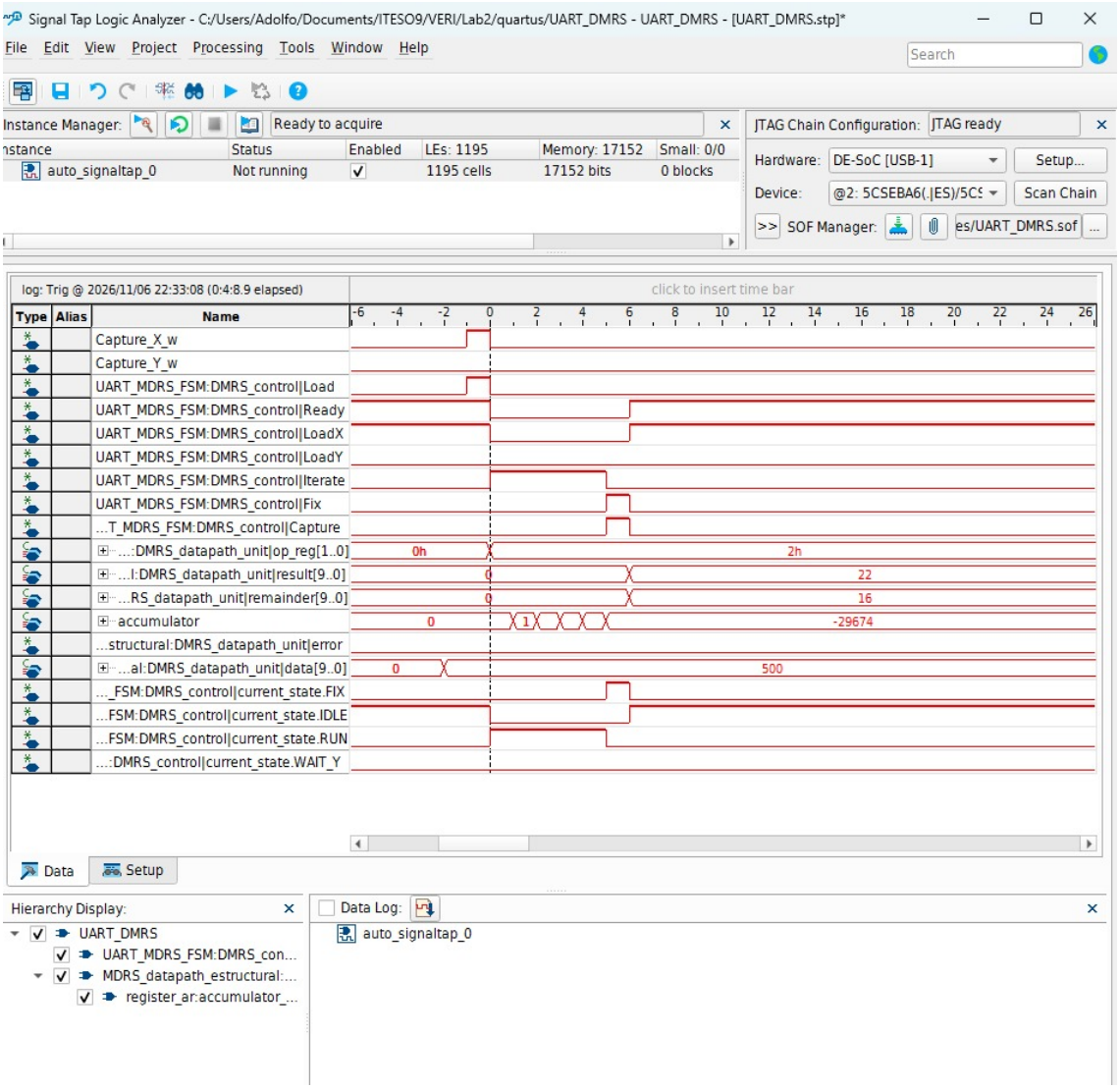
Utilizando el tool de RTL viewer y dentro de la instancia de datapath se tiene la catergorias de registros dentro de la instancia. Para esto Dathapath consta de 64 registros

Síntesis de datapath y RTL.

RTL.



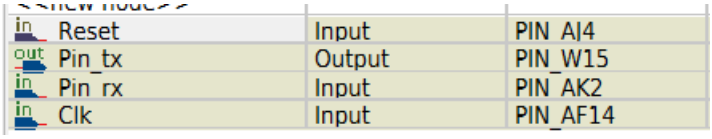
SignalTab.



Implementación.

Mapeo de pines.

Part	Pin	Signal
GPIO		
GPIO[0]	PIN_W15	Pin_tx
GPIO[1]	PIN_AK2	pin_rx
LED		
KEY[0]	PIN_AJ4	Reset
CLK		
CLOCK_50	PIN_AF14	Clock



Reset	Input	PIN_AJ4
Pin_tx	Output	PIN_W15
Pin_rx	Input	PIN_AK2
Clk	Input	PIN_AF14

Capturas de funcionamiento de MDRS.

Multiplicación

>>> m.load(0, 3);m.load(0, 5)	>>> m.load(0, -5);m.load(0, 5)	>>> m.load(0, 200);m.load(0, 100)
>>> m.load(0, 3);m.load(0, 5) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (15, 0, 0, 1, 0, 0) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 1 LoadY = 0 Ready = 1 Error = 0 (15, 0, 1, 0, 1, 0)	>>> m.load(0, -5);m.load(0, 5) Result = 15 (0x000f) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (15, 0, 0, 1, 0, 0) Result = -25 (0x03e7) Remainder = 0 (0x0000) LoadX = 1 LoadY = 0 Ready = 1 Error = 0 (-25, 0, 1, 0, 1, 0)	>>> m.load(0, 200);m.load(0, 100) Result = -25 (0x03e7) Remainder = 0 (0x0000) LoadX = 0 LoadY = 1 Ready = 0 Error = 0 (-25, 0, 0, 1, 0, 0) Result = -1 (0x03ff) Remainder = -1 (0x03ff) LoadX = 1 LoadY = 0 Ready = 1 Error = 1 (-1, -1, 1, 0, 1, 1)

División

>>> m.load(1, 4);m.load(1, 100)

```
>>> m.load(1, 4);m.load(1, 100)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 4, 0, 1, 0, 0)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(0, 4, 1, 0, 1, 0)
```

>>> m.load(1, 47);m.load(1, 0)

```
>>> m.load(1, 47);m.load(1, 0)
Result = 0 (0x0000)
Remainder =  4 (0x0004)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 4, 0, 1, 0, 0)
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(1, 47);m.load(1, 2)

```
>>> m.load(1, 47);m.load(1, 2)
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 0
LoadY = 1
Ready = 0
Error = 1
(-1, -1, 0, 1, 0, 1)
Result = 23 (0x0017)
Remainder =  1 (0x0001)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(23, 1, 1, 0, 1, 0)
```

>>> m.load(1, -125);m.load(1, 5)

```
>>> m.load(1, -125);m.load(1, 5)
Result = 23 (0x0017)
Remainder =  1 (0x0001)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(23, 1, 0, 1, 0, 0)
Result = -25 (0x03e7)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(-25, 0, 1, 0, 1, 0)
```

Raíz

>>> m.load(2, -91)

```
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(2, 89)

```
Result = 9 (0x0009)
Remainder =  8 (0x0008)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(9, 8, 1, 0, 1, 0)
```

>>> m.load(2, 9)

```
Result = 3 (0x0003)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(3, 0, 1, 0, 1, 0)
```

Square

>>> m.load(3, 9)

```
Result = 81 (0x0051)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(81, 0, 1, 0, 1, 0)
```

>>> m.load(3, 100)

```
Result = -1 (0x03ff)
Remainder =  -1 (0x03ff)
LoadX = 1
LoadY = 0
Ready = 1
Error = 1
(-1, -1, 1, 0, 1, 1)
```

>>> m.load(3, -8)

```
Result = 64 (0x0040)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(64, 0, 1, 0, 1, 0)
```

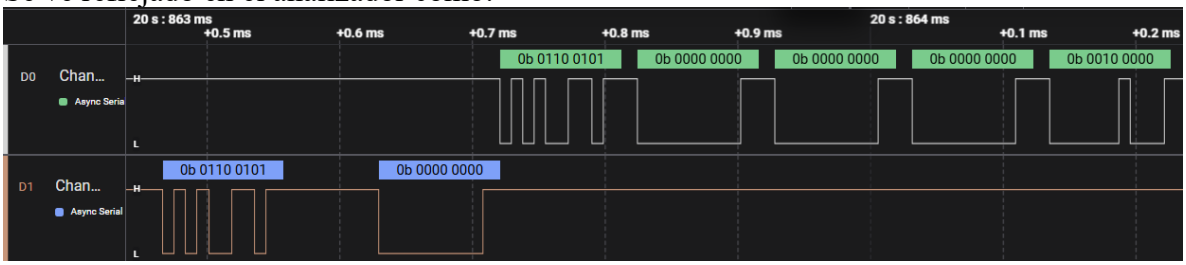
Validación.

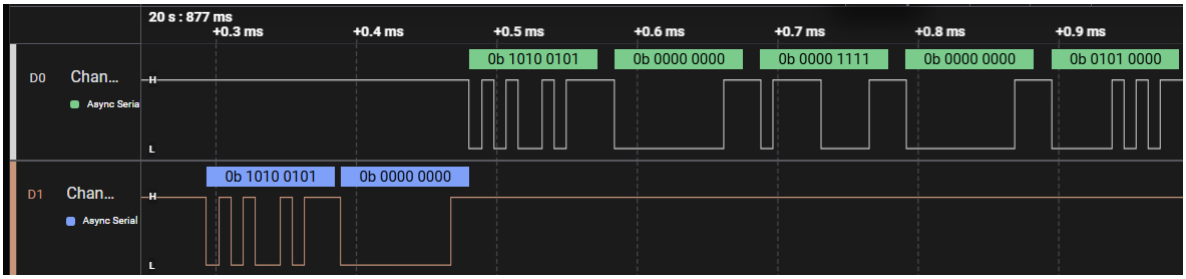
Tomando en cuenta en la estructura en la que se mandan los datos se tiene esto.

BYTE	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
TX_0	data_lower_3_bits			operation		Load	Rsvd	Trigger=1
TX_1	data_higher_7_bits							0
RX_0	data_lower_3_bits			operation		Load	Rsvd	1
RX_1	data_higher_7_bits							0
RX_2	result_lower_8_bits							
RX_3	remainder_lower_6_bits						result_higher_2_bits	
RX_4	error	ready	loadY	loadX	remainder_higher_4_bits			

Enviando un: m.load(0, 3);m.load(0, 5)

Se ve reflejado en el analizador como:





Por lo tanto visto en la estructura:

m.load(0, 3)

	Bit							
BYTE	7	6	5	4	3	2	1	0
Tx_0	0	1	1	0	0	1	0	1
Tx_1	0	0	0	0	0	0	0	0
Rx_0	0	1	1	0	0	1	0	1
Rx_1	0	0	0	0	0	0	0	0
Rx_2	0	0	0	0	0	0	0	0
Rx_3	0	0	0	0	0	0	0	0
Rx_4	0	0	1	0	0	0	0	0

m.load(0, 5)

	Bit							
BYTE	7	6	5	4	3	2	1	0
Tx_0	1	0	1	0	0	1	0	1
Tx_1	0	0	0	0	0	0	0	0
Rx_0	1	0	1	0	0	1	0	1
Rx_1	0	0	0	0	0	0	0	0
Rx_2	0	0	0	0	1	1	1	1
Rx_3	0	0	0	0	0	0	0	0
Rx_4	0	1	0	1	0	0	0	0

Con m.load(0, 3):
Tx_0: data_lower_3_bits = 011, operation = 00 (mult), Load = 1, Rsvd = 0, Trigger = 1
Tx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_0: data_lower_3_bits = 011, operation = 00 (mult), Load = 1, Rsvd = 0, el siempre 1
Rx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_2: result_lower_8_bits = 0000 0000
Rx_3: remainder_lower_6_bits = 00 0000, result_higher_2_bits = 00
Rx_4: error = 0, ready = 0, loadY = 1, loadX = 0, remainder_higher_4_bits = 0000

Con m.load(0, 5):
Tx_0: data_lower_3_bits = 101, operation = 00 (mult), Load = 1, Rsvd = 0, Trigger = 1
Tx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_0: data_lower_3_bits = 101, operation = 00 (mult), Load = 1, Rsvd = 0, el siempre 1
Rx_1: data_lower_7_bits = 0000 000, y el siempre 0
Rx_2: result_lower_8_bits = 0000 1111
Rx_3: remainder_lower_6_bits = 00 0000, result_higher_2_bits = 00
Rx_4: error = 0, ready = 1, loadY = 0, loadX = 1, remainder_higher_4_bits = 0000

Y ya en la terminal esta:

```
>>> m.load(0, 3);m.load(0, 5)
Result = 0 (0x0000)
Remainder =  0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 0, 0, 1, 0, 0)
Result = 15 (0x000f)
Remainder =  0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(15, 0, 1, 0, 1, 0)
```

Result = 0 (0x0000)
Remainder = 0 (0x0000)
LoadX = 0
LoadY = 1
Ready = 0
Error = 0
(0, 0, 0, 1, 0, 0)
Result = 15 (0x000f)
Remainder = 0 (0x0000)
LoadX = 1
LoadY = 0
Ready = 1
Error = 0
(15, 0, 1, 0, 1, 0)

Python Script auto-check (Extra 10%).

Se mantuvo el driver original (115200 baudios, paridad impar, 2 bytes de envío y 5 de respuesta). Se agregó una función de verificación automática contra un modelo de referencia en software.

Los cambios principales son.

wait_bytes() en la clase UART, llamado en _send_receive_cmd.
Espera a que lleguen los 5 bytes, sin la pausa fija de 1 s por comando. La automatización pasa de horas a minutos.

Parámetro quiet en load() y _send_receive_cmd.
Permite silenciar los "printf" durante la automatización. Asi se evita que imprima todos los casos cuando se prueben los todos los casos posibles.

load() retorna (result, remainder, loadX, loadY, ready, error)
El resultado se puede usar desde código y no solo leerlo en el log.

Se agregaron 2 import nuevas.

math	isqrt para la raíz cuadrada entera del modelo
random	Casos aleatorios y mezcla de operaciones

Se agrego también una función nueva. auto(mode=X, n=Y).

Para auto se tienen los siguientes argumentos posibles.

Modo	Que hace
0	Multiplicacion
1	Division
2	Square Root
3	Cuadrado
4	Todas las operaciones que deberían pasar
5	Todas las operaciones que no deberían pasar

N	Efecto
0	Todos los casos del modo seleccionado
>0	n casos al random por operación

Datapath estructural.

Para el extra se hizo una segunda versión del datapath, "MDRS_datapath_estructural", que es la que va en el top. Hace exactamente lo mismo que "MDRS_datapath" pero sin ningún always_comb ni always_ff: todo son instancias de bloques básicos y cables (assign con concatenaciones y compuertas).

Bloques básicos

- register_ar: registro con enable y reset asíncrono activo en 0. Es el único bloque secuencial.

- mux2: multiplexor 2 a 1 de DW bits.

- mux4: multiplexor 4 a 1, armado con tres mux2.

- Sumador_Restador: el mismo de siempre, es la única operación aritmética.

Estos cuatro caen en la lista del spec (registros, multiplexores y operaciones add/sub); las compuertas van como assign con &, |, ^ y ~.

Cómo se pasó de conductual a estructural

La idea fue que el estructural se leyera igual que el conductual: mismas señales, mismo orden. Cada construcción se tradujo así:

Conductual	**Estructural**
if (c) x = a; else x = b;	mux2 con sel = c
case (op_now) con DIV, SQRT y default	mux4 con sel = op_now (in0 e in3 son Booth, igual que el default)
if / else if / else de los operandos del sumador por fase	dos mux2 anidados
case (Product[1:0]) de Booth	compuertas: Accumulate_w = P[1] ^ P[0], Subtract_w = P[1] & ~P[0]
(y_reg == 0), &bits, |bits	compuertas de reducción (~|, &, |)
always_ff con case (op_now)	un cable *_next por registro armado con muxes y un register_ar con su enable
Concatenaciones y rebanadas ({a, b}, acc[9:0])	se quedan igual, son solo cables

Decisiones que se tomaron:

- Los enables salen tal cual del always_ff original: “op_reg” y “x_reg” con save_x; “y_reg” con save_y en división y con save_x o en_step en raíz; “accumulator” con save_x, save_y o en_step; result, remainder y error con capture o en_write_result (la FSM los sube juntos en FIX).

- Cuando no hay ninguna fase activa el sumador calcula el caso de FIX. Como en ese ciclo ningún registro tiene enable no afecta, y se ahorran tres mux de 21 bits.

Resultados

Los 64 flip-flops quedan repartidos en los 7 register_ar y hay un solo Sumador_Restador, que es lo que se ve en el RTL Viewer.

Registro	**Flip-flops**	**Máximo del spec**
accumulator_register	21	2*DW + 1 = 21
x_register	10	DW = 10
y_register	10	DW = 10
result_register	10	DW = 10
remainder_register	10	DW = 10
op_register	2	2
error_register	1	1
Total	64	65 (ready esta en la FSM)

El testbench mdrs_estructural_tb instancia los dos datapaths con la misma FSM y el mismo contador, les manda los mismos estímulos y compara result, remainder, error y op_reg en cada ciclo. 0 diferencias de resultados entre conductual y estructural. Además, pasó el mismo barrido exhaustivo que el conductual (todas las multiplicaciones y divisiones, 2.3 millones de operaciones) y la simulación del top por UART.